

Practical-05

a) Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano producer_consumer.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc producer_consumer.c -o producer_consumer
himasree@LAPTOP-41COT5AA:~/OSLab$ ./producer_consumer
Consumer received 100000 values.
Producer sent 100000 values.
Data transferred: 400000 bytes
Communication time: 0.090638 seconds
Throughput: 4413182.15 bytes/second
himasree@LAPTOP-41COT5AA:~/OSLab$
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define N 100000

int main()
{
    int pipefd[2];
    pid_t pid;
    int data;

    struct timespec start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid == -1)
    {
        perror("fork");
        return 1;
    }
```

```
    // Child process - Consumer
    if (pid == 0)
    {
        close(pipefd[1]); // Close unused write end

        int count = 0;

        while (read(pipefd[0], &data, sizeof(data)) > 0)
        {
            count++;
        }

        close(pipefd[0]);

        printf("Consumer received %d values.\n", count);

        exit(0);
    }

    // Parent process - Producer
    else
    {
        close(pipefd[0]); // Close unused read end

        clock_gettime(CLOCK_MONOTONIC, &start);

        for (int i = 1; i <= N; i++)
        {
            data = i;
```

```

        if (write(pipefd[1], &data, sizeof(data)) == -1)
        {
            perror("write");
            return 1;
        }
    }

    close(pipefd[1]);

    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    double bytes = N * sizeof(int);
    double throughput = bytes / time_taken;

    printf("Producer sent %d values.\n", N);
    printf("Data transferred: %.0f bytes\n", bytes);
    printf("Communication time: %.6f seconds\n", time_taken);
    printf("Throughput: %.2f bytes/second\n", throughput);
}

return 0;
}

```

b) Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```

himasree@LAPTOP-41COT5AA:~/OSLab$ nano ls_grep.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc ls_grep.c -o ls_grep
himasree@LAPTOP-41COT5AA:~/OSLab$ ./ls_grep
Consumer received 100000 values.
Producer sent 100000 values.
Data transferred: 400000 bytes
Communication time: 0.075297 seconds
Throughput: 5312326.34 bytes/second

```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define N 100000

int main()
{
    int pipefd[2];
    pid_t pid;
    int data;

    struct timespec start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid == -1)
    {

```

```

        perror("fork");
        return 1;
    }

    // Child process - Consumer
    if (pid == 0)
    {
        close(pipefd[1]);

        int count = 0;

        while (read(pipefd[0], &data, sizeof(data)) > 0)
        {
            count++;
        }

        close(pipefd[0]);

        printf("Consumer received %d values.\n", count);
        exit(0);
    }

    // Parent process - Producer
    else

```

```

{
    close(pipefd[0]);

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 1; i <= N; i++)
    {
        data = i;

        if (write(pipefd[1], &data, sizeof(data)) == -1)
        {
            perror("write");
            return 1;
        }
    }

    close(pipefd[1]);

    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

```

```

    double bytes = N * sizeof(int);
    double throughput = bytes / time_taken;

    printf("Producer sent %d values.\n", N);
    printf("Data transferred: %.0f bytes\n", bytes);
    printf("Communication time: %.6f seconds\n", time_taken);
    printf("Throughput: %.2f bytes/second\n", throughput);
}

return 0;

```